Applied Soft Computing 11 (2011) 3625–3633

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

A real-integer-discrete-coded particle swarm optimization for design problems

Dilip Datta [a,c,*], José Rui Figueira [b,c]

[a] Department of Mechanical Engineering, National Institute of Technology-Silchar, Silchar 788 010, India
[b] INPL, Ecole des Mines de Nancy, Département de Génie Industriel, Laboratoire LORIA, Parc de Saurupt CS 14 234, 54 042 Nancy Cedex, France
[c] Associate Member of CEG-IST, Center for Management Studies, Instituto Superior Técnico, Technical University of Lisbon, Tagus Park, 2780-990 Porto Salvo, Portugal

ARTICLE INFO

Article history:
Received 5 July 2010
Received in revised form 11 October 2010
Accepted 22 January 2011
Available online 1 February 2011

Keywords:
Binary-coded particle swarm optimization
Real variable
Integer variable
Discrete variable

ABSTRACT

The successful application of *particle swarm optimization* (PSO) to various real-valued problems motivates to develop some integer-coded versions of PSO for working directly with integer and discrete variables of a problem. However, in most of such works, actually a real-valued solution is just converted into a desired integer-valued solution by applying some posterior decoding mechanisms, while some problem-specific integer-coded versions of PSO are proposed in others. In this article, a novel version of PSO of general form is proposed, which can work directly with real, integer and discrete variables of a problem without any conversion. Quite satisfactory performance is obtained by applying the proposed PSO to some classical mechanical design problems of different nature.

1. Introduction

The particle swarm optimization (PSO) is proposed by Kennedy and Eberthart [15], which works with real-valued variables of a problem. However, after its successful application to a wide range of problems, it is now realized that PSO is lacking to work with integer and discrete variables (integer numbers are special cases of discrete numbers having values from the set of natural numbers, i.e., whole numbers with unit difference). To do so, those variables are to be encoded to real-form and then to be decoded back to their original form after the optimization process. Although very simple, in such a process many potential information are likely to be lost in shifting from one search space to another and then to come back to the original one. Therefore, researchers are now trying to develop some integer-coded versions of PSO, which would be able to work directly with integer and discrete variables of a problem. However, it is observed that, instead of an integer-coded PSO, most of those works actually concentrate on developing some efficient encoding and decoding mechanisms for dealing with integer-valued variables in the real-coded PSO. Only a limited number of works are found in which attempts are made for developing some problem-specific integer-coded versions of PSO.

In this article, a new binary-coded PSO of general form is proposed for directly dealing with integer and discrete variables of a problem without any conversion. It is then combined with a real-coded PSO for forming a real-integer-discrete-coded PSO, which can work with any type of variables. Moreover, in order to improve the performance of the proposed PSO, some changes are made to the concept of *global best particle* of the traditional PSO. In this article, the proposed PSO is applied to some mechanical design problems, which are similar with many other design problems. The obtained results depict that the PSO has the potentiality for efficiently solving practical design problems involving different types of decision variables.

The rest of the article is organized as follows: the related specialized literature is briefly reviewed in Section 2, followed by the introduction to the existing real-coded PSO in Section 3. The proposed real-integer-discrete-coded PSO, along with the changes and modifications for improving its performance, is presented in Section 4. The effectiveness of the PSO is presented in Section 5 through its application to some numerical examples. Finally, the article is concluded in Section 6 by addressing the present findings as well as the future scope of the work.

2. Literature review

Kennedy and Eberthart, the original proposers of PSO (Kennedy and Eberthart [15]), propose a binary-coded PSO also for working with discrete variables, in which a sigmoid function is used with a random probability for generating binary-valued (0 or 1) position for a particle from its real-valued velocity component [16]. Laskari et al. [18] propose a discrete PSO, in which a real value is gradually truncated to its nearest integer. In order to solve the job scheduling problem, Liu et al. [21], Tasgetiren et al. [26] apply some decoding

* Corresponding author. Tel.: +91 9435170182.
E-mail addresses: datta_dilip@rediffmail.com, ddatta@nits.ac.in (D. Datta), Jose.Figueira@mines.inpl-nancy.fr (J.R. Figueira).

1568-4946/$ – see front matter © 2011 Elsevier B.V. All rights reserved.
doi:10.1016/j.asoc.2011.01.034

mechanisms for converting a real-valued solution of PSO into an integer-valued permutation. Some discrete versions of PSO, applicable to the job scheduling problem, are proposed in Anghinolfi and Paolucci [1], Kashan and Karimi [14], Liao et al. [19], Sha and Hsu [25], in which the problem information is incorporated in such a way that the positions of a particle of PSO give an integer-valued permutation.

3. The real-coded PSO

PSO is a swarm intelligence-based optimization technique, inspired by the social behavior of bird flocking, in which a swarm of some particles/birds gradually improve their movement towards a piece of food. Unlike many other metaheuristics, such as genetic algorithm and differential evolution, which use random solutions for generating new solutions, PSO uses some previously explored best particles (solutions) for improving a particle in question. This characteristic of PSO increases its probability for generating good particles, and hence, to converge faster to the optimum. At any instant of PSO, a particle changes its velocity and position by exploiting the best position attained so far by itself (*personal best* or *p*-best) or by any other particle of the swarm (*global best* or *g*-best). The mathematical formulation of PSO for a swarm of N particles can be expressed as below:

$$v_j^{(i,t+1)} = \omega v_j^{(i,t)} + c_1 r_{1j}^{(i,t)} \left(\bar{x}_j^{(i,t)} - x_j^{(i,t)} \right) + c_2 r_{2j}^{(i,t)} \left(\bar{\bar{x}}_j^{(t)} - x_j^{(i,t)} \right) \tag{1}$$

$$x_j^{(i,t+1)} = x_j^{(i,t)} + v_j^{(i,t+1)}; \tag{2}$$

where t is the instant counter; $v_j^{(i)}$, $x_j^{(i)}$ and $\bar{x}_j^{(i)}$ are, respectively, the jth dimensional real-valued velocity, position and p-best of the ith particle; $\bar{\bar{x}}_j$ is the jth dimensional g-best of the swarm; ω is an inertia constant; c_1 and c_2 are, respectively, the cognitive and social behavioral factors of the particles; $r_{1j}^{(i)}$ and $r_{2j}^{(i)}$ are two random numbers in the range of]0,1[for the jth dimension of the ith particle. It is to be mentioned that if Eq. (2) is observed carefully, one will find PSO as similar with an exact numerical optimization method, where the solution $\boldsymbol{x}^{(t)}$ is updated as $\boldsymbol{x}^{(t+1)} = \boldsymbol{x}^{(t)} + \delta\boldsymbol{x}^{(t)}$, i.e., the velocity $\boldsymbol{v}$ of PSO resembles with the change in solution $\delta\boldsymbol{x}$ of an exact method. The main difference is that PSO obtains $\boldsymbol{v}$ stochastically, instead of going through any expensive exact computation as done in obtaining $\delta\boldsymbol{x}$. Further, ω, c_1 and c_2, used in Eq. (1) for computing $\boldsymbol{v}$, are similar with move limits to a solution, used in the move limit method [2], that control the amount of $\delta\boldsymbol{x}$.

After updating the position as above, the p-best of a particle and the g-best of the swarm (say, for a minimization problem) can be updated as given below by Eqs. (3) and (4), respectively:

$$\bar{\boldsymbol{x}}^{(i,t+1)} = \begin{cases} \boldsymbol{x}^{(i,t+1)} & \text{if} f\left(\boldsymbol{x}^{(i,t+1)}\right) < f\left(\bar{\boldsymbol{x}}^{(i,t)}\right) \\ \bar{\boldsymbol{x}}^{(i,t)} & \text{otherwise;} \end{cases} \tag{3}$$

$$\bar{\bar{\boldsymbol{x}}}^{(t+1)} = \begin{cases} \bar{\boldsymbol{x}}^{(i,t+1)} & \text{if } \min\left\{ f\left(\bar{\boldsymbol{x}}^{(i,t+1)}\right); i \in \{1,\ldots,N\} \right\} < f\left(\bar{\bar{\boldsymbol{x}}}^{(t)}\right) \\ \bar{\bar{\boldsymbol{x}}}^{(t)} & \text{otherwise;} \end{cases} \tag{4}$$

where $f(\boldsymbol{x})$ is the objective value at $\boldsymbol{x}$.

4. The proposed real-integer-discrete-coded PSO

The only difference between a real-coded PSO and an integer-coded PSO is the data-type of the generated velocity and position of a particle as given by Eqs. (1) and (2), respectively. As mentioned in Section 1, in the general integer-coded versions of PSO, some random schemes are employed for generating an integer-valued position from a real-valued velocity [16,18], while individual problem information is employed in others for directly generating an integer-valued position [1,14,19,25]. However, in the actual sense of an "integer-coded algorithm", only some integer values should be employed to generate a new integer value without any conversion. Such a binary-coded PSO is proposed here, in which based on some logic, only −1, 0 or 1 is assigned to a velocity component of a particle so that a {0,1} binary value can be obtained for its position. This binary-coded PSO is combined with the real-coded PSO, addressed in Section 3, for forming a real-integer-discrete-coded PSO, which can work with any type of variables. The proposed PSO is addressed below in detail.

4.1. Particle representation and initialization

The total number of positional dimensions of a particle is determined from the number and types of variables. Each real variable is represented by a single dimension. The number of dimensions required for an integer variable is determined from the upper limit of the variable, which is obtained as the maximum number of binary bits required to represent its upper limit. It is to be mentioned that a discrete variable is also dealt with as an integer variable, an integer value of which represents the index of its actual discrete value, so that the lower limit of such a variable is 1 and the upper limit is the number of its allowable discrete values.

Accordingly, the total number of positional dimensions of a particle is calculated as below:

$$L = R + \sum_{j=1}^{I} B_j^{(I)} + \sum_{k=1}^{D} B_k^{(D)}; \tag{5}$$

where L is the total number of positional dimensions of the particle; R, I and D are, respectively, the numbers of its real, integer and discrete variables; $B_j^{(I)}$ is the number of binary bits required to represent the jth integer variable; $B_k^{(D)}$ is the number of binary bits required to represent the k th discrete variable.

At the time of decoding an integer variable, its integer value from its binary bits is obtained as follows:

$$x = \sum_{i=1}^{B} 2^{B-i} b_i; \tag{6}$$

where x is the integer value of the variable, B is the number of its binary bits, and b_i is its ith binary value.

Once the total number of positional dimensions of a particle is determined, the swarm is initialized randomly before starting the PSO operations. As seen above, a particle consists of two types of dimensional positions: real dimensions and binary dimensions. A real dimension is initialized by a random real value in the given range of the real variable which is represented by that dimension, while all the binary dimensions are initialized randomly by 0 or 1 with 50% probability. On the other hand, all the velocity components are assigned the initial value of 0.

4.2. Velocity and position of a real variable

Since the basic real-coded PSO is already an established method for continuous search spaces, Eqs. (1) and (2), representing the basic real-coded PSO, are directly applied here also for generating real-valued velocity and position for a real variable.

Table 1
Different cases of Eq. (1) for binary-coded PSO with $x_j^{(i,t)} = 0$.

Column →	$v_j^{(i,t)}$	$\bar{x}_j^{(i,t)}$	$\bar{\bar{x}}_j^{(t)}$	$v_j^{(i,t+1)}$ (allowed value is 0 or 1 only)			
Row ↓	(1)	(2)	(3)	(4)	(5)	(6)	(7)
(1)	0	0	0	0	0	0	0 if $r_1^{(t)} \geq p_m$, 1 otherwise
(2)	0	0	1	$c_2 r_2$	≥0	0 or 1	0 if $r_2^{(t)} \leq 0.5$, 1 otherwise
(3)	0	1	0	$c_1 r_1$	≥0	0 or 1	0 if $r_3^{(t)} \leq 0.5$, 1 otherwise
(4)	0	1	1	$c_1 r_1 + c_2 r_2$	>0	1	1 if $r_4^{(t)} \geq p_m$, 0 otherwise
(5)	1	0	0	w	≥0	0 or 1	0 if $r_5^{(t)} \leq 0.5$, 1 otherwise
(6)	1	0	1	$w + c_2 r_2$	>0	1	1 if $r_6^{(t)} \geq p_m$, 0 otherwise
(7)	1	1	0	$w + c_1 r_1$	>0	1	1 if $r_7^{(t)} \geq p_m$, 0 otherwise
(8)	1	1	1	$w + c_1 r_1 + c_2 r_2$	>0	1	1 if $r_8^{(t)} \geq p_m$, 0 otherwise
(9)	−1	0	0	$-w$	0	0	0 if $r_9^{(t)} \leq 0.5$, 1 otherwise
(10)	−1	0	1	$-w + c_2 r_2$	–	0 or 1	0 if $r_{10}^{(t)} \leq 0.5$, 1 otherwise
(11)	−1	1	0	$-w + c_1 r_1$	–	0 or 1	0 if $r_{11}^{(t)} \leq 0.5$, 1 otherwise
(12)	−1	1	1	$-w + c_1 r_1 + c_2 r_2$	–	0 or 1	0 if $r_{12}^{(t)} \leq 0.5$, 1 otherwise

4.3. Velocity components for generating a binary position

In an integer-coded PSO of general form, usually some random schemes are applied for generating integer-valued positions from real-valued velocity components [16,18]. Instead of such random schemes, some logic are developed here for assigning some particular values to the velocity components of a particle, which will generate only binary-valued positions for the particle.

Since the possible values of $x_j^{(i,t)}$ and $x_j^{(i,t+1)}$ in Eq. (2) are 0 or 1 only, the following conditions are hold:

$$x_j^{(i,t+1)} = \begin{cases} 0; & \text{if } \left(x_j^{(i,t)}, v_j^{(i,t+1)}\right) = (0,0) \text{ or } (1,-1) \\ 1; & \text{if } \left(x_j^{(i,t)}, v_j^{(i,t+1)}\right) = (0,1) \text{ or } (1,0) \end{cases}$$
$$\text{Or,} \quad x_j^{(i,t)} = \begin{cases} 0 \Rightarrow v_j^{(i,t+1)} = 0, 1 \\ 1 \Rightarrow v_j^{(i,t+1)} = 0, -1 \end{cases} \tag{7}$$

That is, according to Eq. (7), the values of $v_j^{(i,t+1)}$, as well as of $v_j^{(i,t)}$, in Eq. (1) should be 0, 1 or −1 only. Therefore, against each of the two values of $x_j^{(i,t)}$, there exist 12 distinct combinations of the values of $v_j^{(i,t)}$, $\bar{x}_j^{(i,t)}$ and $\bar{\bar{x}}_j^{(t)}$ ($\bar{x}_j^{(i,t)}$ and $\bar{\bar{x}}_j^{(t)}$ are also {0,1} variables), which are given in columns (1)–(3) in Tables 1 and 2. The corresponding expressions of $v_j^{(i,t+1)}$ are given in column (4), which are to be mapped to 0 or 1 in Table 1, and to 0 or −1 in Table 2. For this, based on its possible value, each $v_j^{(i,t+1)}$ is first related with 0 as ">0" or "≥ 0" in Table 1, and as "<0" or "≤ 0" in Table 2, which is given in column (5). It is to be noted that the value of $v_j^{(i,t+1)}$ in row (9) of Table 1 must be 0 as it cannot be negative for $x_j^{(i,t)} = 0$. Similarly, its value in row (8) of Table 2 also must be 0 as it cannot be positive for $x_j^{(i,t)} = 1$. On the other hand, since $v_j^{(i,t+1)}$ in rows (10)–(12) of Table 1 and in rows (5)–(7) of Table 2 consist of both positive and negative terms, these could not be related with 0 as above. Finally, based on the following assumptions, the relations of column (5) are mapped to 0, 1 or −1:

$$\left.\begin{array}{l} v_j^{(i,t+1)} > 0 \Rightarrow v_j^{(i,t+1)} = 1 \\ v_j^{(i,t+1)} < 0 \Rightarrow v_j^{(i,t+1)} = -1 \\ v_j^{(i,t+1)} \geq 0, \text{ or no relation in Table 1} \Rightarrow v_j^{(i,t+1)} = 0 \text{ or } 1 \\ v_j^{(i,t+1)} \leq 0, \text{ or no relation in Table 2} \Rightarrow v_j^{(i,t+1)} = 0 \text{ or -1.} \end{array}\right\} \tag{8}$$

The above mapping of $v_j^{(i,t+1)}$ are given in column 6 of Tables 1 and 2. However, these simple and fixed mapping may suffer from the following two major drawbacks:

1. The particles may lose diversity among them, resulting the swarm to converge at a sub-optimal point, and
2. The mapping may not always be true, i.e., in some cases $v_j^{(i,t+1)}$ may take other alternative values, instead of the mapped ones.

In order to avoid such situations, some randomness is imposed to $v_j^{(i,t+1)}$, in which a value of $v_j^{(i,t+1)}$ in column (6) is assumed to be true if a random probability $r_k^{(t)}$ ($k \in \{1, \ldots, 12\}$) is at least equal to a predefined probability p_m (named as the *mutation probability*), otherwise $v_j^{(i,t+1)}$ will take its alternative value. In the case of double options, each value is considered with 50% probability. The final values of $v_j^{(i,t+1)}$, along with the corresponding randomness conditions, are given in column (7).

In the above mapping, the parameters w, c_1, c_2, r_1 and r_2 of Eq. (1) are not required explicitly, but the mapping is done with the presumption that they are non-negative. However, in most of the cases, the new user-defined mutation probability p_m is introduced. Extensive empirical studies have shown that better results are obtained for $p_m \leq 15\%$.

4.4. Updating the personal best and global best particles

Eqs. (3) and (4) update the p-best of a particle and the g-best of the swarm, based on the objective values of the particles (solutions). These are independent of the data-type of individual velocity or position of any particle, as well as independent of the types of PSO used. Therefore, these equations are applied here also, without any modification, for updating the p-best and g-best particles of the proposed real-integer-discrete-coded PSO.

5. Numerical experiments and discussions

The proposed PSO is implemented in C programming language. In order to demonstrate its effectiveness, four mechanical design problems of different nature are considered, which are similar with many other design problems, and studied by many researchers for evaluating performances of different optimization techniques. The considered four problems are design of a gear train, design of a compression coil spring, design of a pressure vessel, and design of a welded beam. The problems involve different types of variables (real, integer, and/or discrete), as well as different issues related with a real-life design environment. As the problems are clearly defined, fairly easy to understand, and of similar nature even with many complicated large-size design problems, they form a suitable basis for investigating the performance of an optimization technique. Table 3 lists the works in which these problems are studied for this purpose.

Table 2
Different cases of Eq. (1) for binary-coded PSO with $x_j^{(i,t)} = 1$.

Column →	$v_j^{(i,t)}$	$\bar{x}_j^{(i,t)}$	$\bar{\bar{x}}_j^{(t)}$	$v_j^{(i,t+1)}$ (allowed value is 0 or −1 only)			
Row ↓	(1)	(2)	(3)	(4)	(5)	(6)	(7)
(1)	0	0	0	$-c_1r_1 - c_2r_2$	<0	−1	−1 if $r_1^{(t)} \geq p_m$, 0 otherwise
(2)	0	0	1	$-c_1r_1$	≤0	0 or −1	0 if $r_2^{(t)} \leq 0.5$, −1 otherwise
(3)	0	1	0	$-c_2r_2$	≤0	0 or −1	0 if $r_3^{(t)} \leq 0.5$, −1 otherwise
(4)	0	1	1	0	0	0	0 if $r_4^{(t)} \geq p_m$, −1 otherwise
(5)	1	0	0	$w - c_1r_1 - c_2r_2$	–	0 or −1	0 if $r_5^{(t)} \leq 0.5$, −1 otherwise
(6)	1	0	1	$w - c_1r_1$	–	0 or −1	0 if $r_6^{(t)} \leq 0.5$, −1 otherwise
(7)	1	1	0	$w - c_2r_2$	–	0 or −1	0 if $r_7^{(t)} \leq 0.5$, −1 otherwise
(8)	1	1	1	w	0	0	0 if $r_8^{(t)} \geq p_m$, −1 otherwise
(9)	−1	0	0	$-w - c_1r_1 - c_2r_2$	<0	−1	−1 if $r_9^{(t)} \geq p_m$, 0 otherwise
(10)	−1	0	1	$-w - c_1r_1$	<0	−1	−1 if $r_{10}^{(t)} \geq p_m$, 0 otherwise
(11)	−1	1	0	$-w - c_2r_2$	<0	−1	−1 if $r_{11}^{(t)} \geq p_m$, 0 otherwise
(12)	−1	1	1	$-w$	≤0	0 or −1	0 if $r_{12}^{(t)} \leq 0.5$, −1 otherwise

Since the performance of any numerical optimizer may vary with its different user-defined parameter settings, each of the four problems is solved here 30 times for analyzing average performance of the proposed PSO. The performance of the PSO is evaluated in terms of number of function evaluations required in obtaining the reported best solutions. It is to be mentioned that no computational issue is discussed in any publication used here for comparing the performances of the proposed PSO.

In each run, the mutation probability p_m for binary variables is assigned the initial value of 15%. For handling real variables, the assigned initial values of the inertia constant ω, cognitive behavioral factor c_1, and social behavioral factor c_2 are 1.0, 1.0 and 2.0, respectively. However, instead of working with a single fixed value, each of p_m, ω, c_1 and c_2 is made instant-wise self-adaptive in a range from zero to its assigned initial value. Such self-adaptation of a parameter reduces the dependency of an algorithm on that parameter, if any [5]. A value is made self-adaptive through the *polynomial mutation of a real number* [9], in which, for a random number r in the range of]0,1[and a given polynomial distribution index $\eta > 0$, a real number x is evolved in the range of $[x^{(l)}, x^{(u)}]$ as given below:

Table 3
A survey of the works in which the considered problems are studied.

Problem	Investigated optimization techniques
Gear train (Section 5.1)	Branch-and-bound method through sequential quadratic programming [24]. Sequential linearization algorithm [22]. Augmented Lagrange multiplier based method [13]. Simulated annealing algorithm [29]. Genetic algorithm [10,20]. Hybrid swarm intelligence approach [12].
Coil spring (Section 5.2)	Branch-and-bound method through sequential quadratic programming [24]. Genetic algorithm [3,10,27]. Hybrid swarm intelligence approach [12]. Decoding-based differential evolution [17].
Pressure vessel (Section 5.3)	Branch-and-bound method through sequential quadratic programming [24]. Augmented Lagrange multiplier based method [13]. Genetic algorithm [4,7].
Welded beam (Section 5.4)	Genetic algorithm [10].

$$\left.\begin{aligned} x &\leftarrow x + \left(x^{(u)} - x^{(l)}\right) d_q \\ \text{where } d_q &= \left[2r + (1-2r)\left(\frac{x^{(u)} - x}{x^{(u)} - x^{(l)}}\right)^{\eta+1}\right]^{\frac{1}{\eta+1}} - 1, \quad \text{if } r < 0.5 \\ &= 1 - \left[2(1-r) + (2r-1)\left(\frac{x - x^{(l)}}{x^{(u)} - x^{(l)}}\right)^{\eta+1}\right]^{\frac{1}{\eta+1}}, \quad \text{otherwise.} \end{aligned}\right\} \tag{9}$$

The distribution index η in Eq. (9) is assigned a random value, in the range of [25,45], in different runs of a problem. Similarly, the PSO swarm size in different runs is randomly fixed in the range of [50,100]. In the case of execution time, each run is allowed to be continued for a maximum number of 1000 generations. However, a run is terminated in between when no improvement in the best objective value is noticed.

Moreover, the *penalty-parameter-less constraint handling approach*, proposed by Deb [8], is applied here for working with infeasible solutions. In this approach, all the infeasible solutions are first made inferior to any feasible solution through a fitness function, and then all the solutions are treated as feasible solutions only. On the other hand, a local search scheme is applied to the g-best particle in order to avoid any local optimum by exploring its neighborhood. In this scheme, a binary position of a particle is altered, either from 0 to 1 or from 1 to 0, with a small random probability in the range of]0,0.05[. Similarly, a real position is also changed by applying Eq. (9).

Besides the proposed PSO, the considered problems are also solved by the binary-coded PSO of Kennedy and Eberthart [16] for comparison purpose. The implementation is similar with the real-integer-discrete-coded PSO, proposed in Section 4, except that the binary-position generator of Section 4.3 is replaced by that of Kennedy and Eberthart [16], in which binary-positions are generated as below:

$$x_j^{(i,t+1)} = \begin{cases} 1, & \text{if } \; r \leq \dfrac{1}{1 + e^{-v_j^{(i,t+1)}}} \\ 0, & \text{otherwise;} \end{cases} \tag{10}$$

where $v_j^{(i,t+1)}$ and $x_j^{(i,t+1)}$ are, respectively, the jth dimensional real-valued velocity and integer-valued (0 or 1) position of the ith particle at $(t+1)$th instant, and r is a random probability in the range of]0,1[. Different parameter settings for this PSO are also kept the same as above, except that no local search is applied to the g-best particle as done in the case of the PSO proposed here.

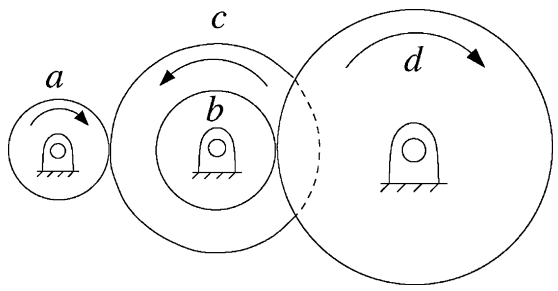

Fig. 1. The gear train design problem.

5.1. Gear train design

The problem involves the optimization of the gear ratio of a compound gear train, so that a desired motion/power can be transmitted from one shaft to another. As shown in Fig. 1, the example gear train arrangement consists of two pairs of gearwheels, *a*–*c* and *b*–*d*, where *a* and *b* are driving gearwheels, and *c* and *d* are driven gearwheels. The overall gear ratio of the arrangement can be expressed as below:

$$\text{Gear ratio} = \frac{\prod(\text{Numbers of teeth on driving gearwheels})}{\prod(\text{Numbers of teeth on driven gearwheels})} = \frac{z_a z_b}{z_c z_d}, \tag{11}$$

where z_a, z_b, z_c and z_d denote the numbers of teeth on the gears *a*, *b*, *c* and *d*, respectively. It is required to determine the values of z_a, z_b, z_c and z_d so that a gear ratio, as close as possible to 1/6.931, can be obtained. The only constraint in the problem is that the number of teeth on any gear should be in the range of [12,60]. Accordingly, the optimization problem can be formulated as follows:

$$\left.\begin{array}{lll} \text{Determine} & \boldsymbol{x} = & (z_a, z_b, z_c, z_d) \\ \text{to minimize} & f(\boldsymbol{x}) & = \left(\frac{1}{6.931} - \frac{z_a z_b}{z_c z_d}\right)^2 \\ \text{subject to :} & g(\boldsymbol{x}) & \equiv 12 \le z_a, z_b, z_c, z_d \le 60 \\ & & z_a, z_b, z_c \text{ and } z_d \text{ are integers.} \end{array}\right\} \tag{12}$$

Although seems simple, the problem becomes complicated/expensive to many algorithms as its all four variables are integers. As per the formulation of the proposed PSO, 6 binary bits are required to represent each variable, thus the total number of positional dimensions of a particle becomes 24 to represent all the four variables. Applying the proposed PSO, the known best solution of the problem could be obtained in all of its 30 runs. The PSO of Kennedy and Eberthart [16] is also successful in obtaining the known best solution of the problem. In fact, no further improvement is possible as this known best solution is the global optimum of the problem [10]. A comparison of the results obtained by different optimization techniques is presented in Table 4. It is to be mentioned that, against the best objective value, four distinct sets of variable values are obtained in different runs, or even in different solutions of a single run, depicting that the problem has multiple optimum solutions. The obtained variable sets are (z_a, z_b, z_c, z_d) = (16,19,43,49), (19,16,43,49), (16,19,49,43) and (19,16,49,43).

In the case of computational cost, the numbers of function evaluations are found varying in the range of [300,2600] over the 30 runs of the proposed PSO, with the mean, median and standard deviation of 1076.67, 1000 and 23.88, respectively. In contrast, the number of function evaluations required by the PSO of Kennedy and Eberthart [16] was 19000, which is quite higher than those required by the proposed PSO.

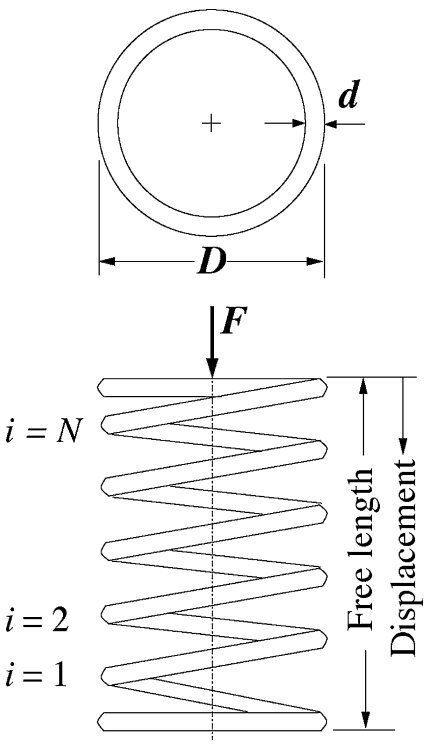

Fig. 2. The compression coil spring design problem.

5.2. Compression coil spring design

This problem involves the design of a coil spring, which is to support a constant axial compressive load. It is required to minimize the wire volume of the spring (minimum weight) so that it can support a given load without failure. As shown in Fig. 2, there are three design variables: the real-valued outside diameter of the spring (*D*), the integer-valued number of spring coils (*N*),

Table 4
Optimal solutions for the gear train design problem.

Variables/ functions	Sandgren [24]	Loh and Papalambros [22]	Zhang and Wang [29]	Lin et al. [20]	Guo et al. [12]	PSO of Kennedy and Eberthart [16]	Proposed PSO
z_a	18	19	30	19	16	16	16
z_b	22	16	15	16	19	19	19
z_c	45	42	52	43	43	43	43
z_d	60	50	60	49	49	49	49
$f(\boldsymbol{x})$	5.7×10^{-6}	0.23×10^{-6}	2.4×10^{-9}	2.7×10^{-12}	2.7×10^{-12}	2.7×10^{-12}	2.7×10^{-12}
Gear ratio	0.146666	0.144762	0.144231	0.144281	0.144281	0.144281	0.144281
Error (%)	1.65	0.334	0.033	0.0011	0.0011	0.0011	0.0011

Table 5
Allowable wire diameters (discrete values of d).

Allowable wire diameters (in.)						
0.0090	0.0095	0.0104	0.0118	0.0128	0.0132	0.0140
0.0150	0.0162	0.0173	0.0180	0.0200	0.0230	0.0250
0.0280	0.0320	0.0350	0.0410	0.0470	0.0540	0.0630
0.0720	0.0800	0.0920	0.1050	0.1200	0.1350	0.1480
0.1620	0.1770	0.1920	0.2070	0.2250	0.2440	0.2630
0.2830	0.3070	0.3310	0.3620	0.3940	0.4375	0.5000

and the discrete-valued spring wire diameter (d). Accordingly, the optimization problem is formulated as below:

$$\begin{aligned}
&\text{Determine} && \boldsymbol{x} = && (D, N, d)\\
&\text{to minimize} && f(\boldsymbol{x}) = && \frac{1}{4}\pi^2 D d^2 (N+2)\\
&\text{subject to :} && g_1(\boldsymbol{x}) \equiv && \frac{8cKF_{\max}}{\pi d^2} - S \le 0\\
& && g_2(\boldsymbol{x}) \equiv && l - l_{\max} \le 0\\
& && g_3(\boldsymbol{x}) \equiv && d_{\min} - d \le 0\\
& && g_4(\boldsymbol{x}) \equiv && (D+d) - D_{\max} \le 0\\
& && g_5(\boldsymbol{x}) \equiv && 3.0 - c \le 0\\
& && g_6(\boldsymbol{x}) \equiv && \delta_p - \delta_{pm} \le 0\\
& && g_7(\boldsymbol{x}) \equiv && \delta_p + \frac{F_{\max} - F_p}{k} + 1.05(N+2)d - l \le 0\\
& && g_8(\boldsymbol{x}) \equiv && \delta_w - \frac{F_{\max} - F_p}{k} \le 0\\
& && && N \text{ is integer and } d \text{ is discrete;}
\end{aligned} \tag{13}$$

where

$$c = \frac{D}{d}$$
$$K = \frac{4c-1}{4c-4} + \frac{0.615}{c}$$
$$k = \frac{Gd}{8Nc^3}$$
$$\delta_p = \frac{F_p}{k}$$
$$l = \frac{F_{\max}}{k} + 1.05(N+2)d$$

The supplied numerical data for the problem are given below along with the allowable discrete values of the spring wire diameter (d) in Table 5.

$F_{\max}$ = Maximum working load = 1000.0 lb
S = Maximum allowable shear stress = 189000.0 psi
$l_{\max}$ = Maximum free length = 14.0 in.
$d_{\min}$ = Minimum wire diameter = 0.2 in.
$D_{\max}$ = Maximum outside spring diameter = 3.0 in.
F_p = Preload compression force = 300.0 lb
δ_{pm} = Maximum allowable deflection under preload = 6.0 in.
δ_w = Deflection from preload position to maximum load position = 1.25 in.
G = Shear modulus of the material = 11.5×10^6 psi.

Combining constraints $g_3(\boldsymbol{x})$–$g_5(\boldsymbol{x})$, the limits of D and N are fixed as [0.6,3.0] in. and [1,70], respectively. Since d is a discrete variable with 42 allowable values given in Table 5, its integer limits are set automatically as [1,42].

As per the formulation of the proposed PSO, the numbers of dimensional positions required for D, N and d are, respectively, 1 real bit, 7 binary bits, and 6 binary bits, thus the total number of positional dimensions of a particle becomes 14 to represent the three variables. Applying the proposed PSO, fixed values of N = 9 and d = 0.283 in. are obtained in each of the 30 runs of this problem. The only variation is obtained in the values of D, which vary in the range of [1.223041,1.223060] in. The corresponding objective values vary in the range of [2.658559,2.658599] in.^{3}. In the case of the PSO of Kennedy and Eberthart [16], the obtained best variable values are D = 1.223047 in., N = 9 and d = 0.283 in., resulting the best objective value of 2.658573 in.^{3}, which is marginally poorer than that given by the proposed PSO. A comparison of the best results obtained by different optimization techniques is presented in Table 6, where it is observed that the solution produced by the proposed PSO is equally as good as the best solution reported in the literature.

In the case of computational cost, the numbers of function evaluations are found varying in the range of [4784,98992] over the 30 runs of the proposed PSO, with the mean, median and standard deviation of 51696.27, 40726 and 172.43, respectively. The number of function evaluations required by the PSO of Kennedy and Eberthart [16] was 650600, which is quite higher than those required by the proposed PSO.

5.3. Pressure vessel design

This is the problem of designing a compressed air storage tank having a minimum volume of 750 ft^3 under some dimensional limitations. As shown in Fig. 3, the cylindrical pressure vessel is capped at both ends by hemispherical heads. It is required to minimize the manufacturing cost of the pressure vessel, which includes the cost

Table 6
Optimal solutions for the compression coil spring design problem.

Variables/ functions	Sandgren [24]	Chen and Tsao [3]	Wu and Chow [27]	Guo et al. [12]	Lampinen and Zelinka [17]	PSO of Kennedy and Eberthart [16]	Proposed PSO
D (in.)	1.180701	1.2287	1.227411	1.223	1.223041	1.223047	1.223041
N	10	9	9	9	9	9	9
d (in.)	0.283	0.283	0.283	0.283	0.283	0.283	0.283
$-g_1(\boldsymbol{x})$	543.09	415.969	550.993	1008.81	1008.8114	1008.1457	1008.8059
$-g_2(\boldsymbol{x})$	8.8187	8.9207	8.9264	8.946	8.94564	8.945608	8.945635
$-g_3(\boldsymbol{x})$	0.08298	0.0830	0.0830	0.083	0.08300	0.083000	0.083000
$-g_4(\boldsymbol{x})$	1.8193	1.7713	1.7726	1.77696	1.77696	1.493953	1.493959
$-g_5(\boldsymbol{x})$	1.1723	1.3417	1.3371	1.32170	1.32170	1.321722	1.321700
$-g_6(\boldsymbol{x})$	5.4643	5.4568	5.4585	5.4643	5.46429	5.464277	5.464286
$-g_7(\boldsymbol{x})$	0.0	0.0	0.0	0.0	0.0	0.000000	0.000000
$-g_8(\boldsymbol{x})$	0.0	0.0174	0.0134	0.0	0.0	0.000019	0.000010
$f(\boldsymbol{x})$ (in^3)	2.7995	2.6709	2.6681	2.659	2.65856	2.658573	2.658559

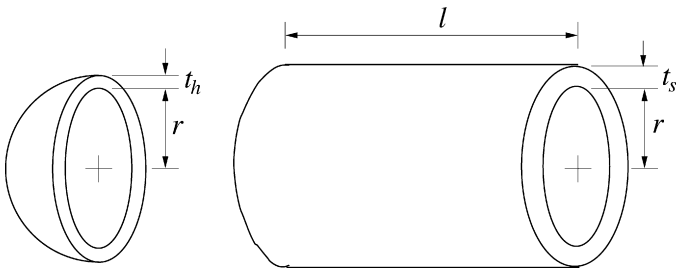

Fig. 3. The pressure vessel design problem.

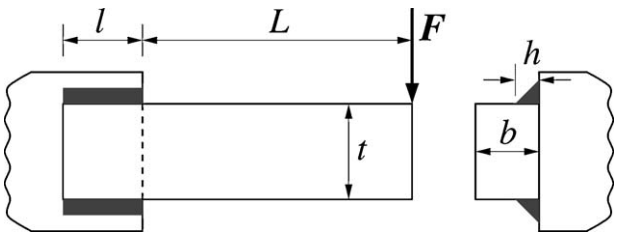

Fig. 4. The welded beam design problem.

of material, welding and forming. The problem involves four design variables: the length of the cylindrical section (l), inner radius of the shell (r), thickness of the shell (t_s), and thickness of the head (t_h). t_s and t_h are discrete variables in multiples of 0.0625 in., which are available thicknesses of rolled steel plates by which the pressure vessel is to be fabricated, while l and r are real variables. Accordingly, the optimization problem is formulated as follows:

$$\left.\begin{array}{lll} \text{Determine} & \boldsymbol{x} = & (l, r, t_s, t_h) \\ \text{to minimize} & f(\boldsymbol{x}) = & 0.6224 t_s r l + 1.7781 t_h r^2 + 3.1661 t_s^2 l + 19.84 t_s^2 r \\ \text{subject to :} & g_1(\boldsymbol{x}) \equiv & 0.0193 r - t_s \le 0 \\ & g_2(\boldsymbol{x}) \equiv & 0.00954 r - t_h \le 0 \\ & g_3(\boldsymbol{x}) \equiv & 1296000 - \pi r^2 l - \frac{4}{3}\pi r^3 \le 0 \\ & g_4(\boldsymbol{x}) \equiv & l - 240 \le 0 \\ & & t_s \text{ and } t_h \text{ are discrete.} \end{array}\right\} \quad (14)$$

The lower limit of l is fixed as 20 in., and hence its range of variation becomes [20,240] in. on taking the constraint $g_4(\boldsymbol{x})$ into account. Once the range of l is fixed, the ranges of r, t_s and t_h are automatically fixed through the constraints $g_3(\boldsymbol{x})$, $g_1(\boldsymbol{x})$ and $g_2(\boldsymbol{x})$, respectively. Thus, the range of variation of r becomes [37.7,63] in. On consideration of only allowed discrete values for t_s and t_h in multiples of 0.0625 in., their ranges of variations come out to be [0.6875,1.25] in. and [0.3125,0.625] in., respectively.

As per the formulation of the proposed PSO, the numbers of dimensional positions required for l, r, t_s and t_h are, respectively, 1 real bit, 1 real bit, 4 binary bits to represent 10 discrete values, and 3 binary bits to represent 6 discrete values, thus the total number of positional dimensions of a particle becomes 9 to represent the four variables. Applying the proposed PSO to this problem, much better solutions than the previously known best solution are obtained in all of the 30 runs, where the overall range of variation in the objective values is [5850.38376,5850.591797] only. In this problem, the performance of the PSO of Kennedy and Eberthart [16] was not satisfactory as it could not obtain even the previously known best solution. A comparison of the best results obtained by different optimization techniques is presented in Table 7.

In the case of computational cost, the numbers of function evaluations are found varying in the range of [31436,124968] over the 30 runs of the proposed PSO, with the mean, median and standard deviation of 82622.67, 84060 and 167.5, respectively. The number of function evaluation required by the PSO of Kennedy and Eberthart [16] in obtaining its best solution was 3815800, which is also quite higher than those required by the proposed PSO.

5.4. Welded beam design

In this problem, a rectangular beam is to be welded to a rigid support as a cantilever beam to carry a certain load without failure. There is an option for the beam to be welded either on two opposite sides or on all four sides of its rectangular cross-section. The beam may be of any of the four materials of steel, cast iron, aluminum, and brass. It is required to design the system so as to minimize the overall fabrication cost. As shown in Fig. 4, apart from the two decision variables related with the choice of welding and beam material, the problem involves two pairs of design variables, each pair related with the dimensions of the weld and beam. The design variables related with the weld are its thickness (h) and length (l), and those related with the beam are its width (t) and thickness (b), where h, t and b are discrete variables in multiples of 0.0625 in. Accordingly, if the beam is to support a constant load of F at a fixed length of L, the optimization problem can be formulated as below:

$$\left.\begin{array}{lll} \text{Determine} & \boldsymbol{x} = & (x_1, x_2, h, t, b, l) \\ \text{to minimize} & f(\boldsymbol{x}) = & (1 + c_1)(x_1 t + l) h^2 + c_2 t b (l + L) \\ \text{subject to :} & g_1(\boldsymbol{x}) \equiv & \sigma(\boldsymbol{x}) - S \le 0 \\ & g_2(\boldsymbol{x}) \equiv & F - P(\boldsymbol{x}) \le 0 \\ & g_3(\boldsymbol{x}) \equiv & \delta(\boldsymbol{x}) - \delta_{\max} \le 0 \\ & g_4(\boldsymbol{x}) \equiv & \tau(\boldsymbol{x}) - 0.577 S \le 0 \\ & & x_1 \in \{0, 1\} \\ & & x_2 \in \{0, 1, 2, 3\} \\ & & h, t \text{ and } b \text{ are discrete,} \end{array}\right\} \quad (15)$$

where

$$\sigma(\boldsymbol{x}) = \frac{6FL}{bt^2}$$

$$P(\boldsymbol{x}) = \frac{4.013 t b^3 \sqrt{EG}}{6L^2}\left[1 - \frac{t}{4L}\sqrt{\frac{E}{G}}\right]$$

$$\delta(\boldsymbol{x}) = \frac{4FL^3}{Et^3 b}$$

$$\tau(\boldsymbol{x}) = \sqrt{(\tau')^2 + (\tau'')^2 + 2\tau'\tau'' \cos\theta}$$

$$\tau' = \frac{F}{A}$$

$$\tau'' = \frac{F\left(L + l/2\right) R}{J}$$

$$x_1 = 0 : \begin{cases} A = \sqrt{2} h l \\ J = \sqrt{2} h l \left[\dfrac{(h+t)^2}{4} + \dfrac{l^2}{12}\right] \\ R = \dfrac{1}{2}\sqrt{l^2 + (h+t)^2} \\ \cos\theta = \dfrac{l}{2R} \end{cases}$$

$$x_1 = 1 : \begin{cases} A = \sqrt{2} h (t + l) \\ J = \sqrt{2} h l \left[\dfrac{(h+t)^2}{4} + \dfrac{l^2}{12}\right] + \sqrt{2} h t \left[\dfrac{(h+l)^2}{4} + \dfrac{t^2}{12}\right] \\ R = \max\left\{\dfrac{1}{2}\sqrt{l^2 + (h+t)^2}, \dfrac{1}{2}\sqrt{t^2 + (h+l)^2}\right\} \\ \cos\theta = \begin{cases} \dfrac{l}{2R}, & \text{if } \ l < t \\ \dfrac{t}{2R}, & \text{otherwise.} \end{cases} \end{cases}$$

In Eq. (15), x_1 represents the type of weld, where $x_1 = 0$ is used for two-sided welding and $x_1 = 1$ is for four-sided welding. Similarly, x_2 indicates the type of material to be used for the beam, where $x_2 = 0$ means steel, $x_2 = 1$ means cast iron, $x_2 = 2$ means aluminum and $x_2 = 3$ means brass. The limits of h, t, b and l are fixed as [0.0625,2.0] in., [2.0,20.0] in., [0.0625,2.0] in. and [1.0,10.0] in., respectively. The given values of F, L and $\delta_{\max}$ are 6000 lb, 14 in. and 0.25 in., respectively. The values of material properties S, E and G,

Table 7
Optimal solutions for the pressure vessel design problem.

Variables/ functions	Sandgren [24]	Kannan and Kramer [13]	Guo et al. [12]	Deb [7]	Coello [4]	PSO of Kennedy and Eberthart [16]	Proposed PSO
l (in.)	106.72	43.690	43.70	112.679	177.080	184.6641	221.365548
r (in.)	48.67	58.291	58.29	48.329	42.0939	41.7165	38.860099
t_s (in.)	1.125	1.125	1.125	0.9375	0.8750	0.81250	0.75000
t_h (in.)	0.625	0.625	0.625	0.5000	0.5000	0.43750	0.37500
$-g_1(\boldsymbol{x})$	0.179	-1.6×10^{-5}	3.0×10^{-6}	0.00475	0.00009	0.007372	0.000000
$-g_2(\boldsymbol{x})$	0.1578	0.068904	0.0689	0.03894	0.03592	0.039525	0.004275
$-g_3(\boldsymbol{x})$	3.0	21.220104	69.24	3652.88	2156.84	17692.83	0.000190
$-g_4(\boldsymbol{x})$	133.28	196.31	196.30	127.321	62.9195	55.3359	18.634452
$f(\boldsymbol{x})$	7982.5	7198.0428	7197.9	6410.38	6069.33	6181.81	5850.38376

Table 8
Material properties and cost terms for the welded beam design problem.

x_2	Material	S (kpsi)	E (Mpsi)	G (Mpsi)	c_1	c_2
0	Steel	30	30	12	0.1047	0.0481
1	Cast iron	8	14	6	0.0489	0.0224
2	Aluminum	5	10	4	0.5235	0.2405
3	Brass	8	16	6	0.5584	0.2566

and those of cost terms c_1 and c_2, corresponding to different values of x_2, are given in Table 8.

Some simplified forms of the problem, for a fixed type of welding of a beam of a given material (two-sided welding of a steel beam), as well as relaxing h, t and b to be real variables, are studied in many works (Coello [4], Deb [6], Deb and Srinivasan [11], Ragsdell and Phillips [23], Yokota et al. [28]). Its full form, as given by Eq. (15), is studied by Deb and Goyal [10] only.

As per the formulation of the proposed PSO, the numbers of dimensional positions required for x_1, x_2, h, t, b and l are 1 binary bit, 3 binary bits, 6 binary bits to represent 32 discrete values, 9 binary bits to represent 289 discrete values, 6 binary bits to represent 32 discrete values, and 1 real bit, respectively. Thus, the total number of positional dimensions of a particle becomes 26 to represent the six variables.

The comparison of the best solutions obtained by Deb and Goyal [10], the PSO of Kennedy and Eberthart [16], and the proposed PSO is given in Table 9, where it is observed that the solutions of Deb and Goyal [10] and the proposed PSO vary marginally in the values of l only, resulting a slightly poorer objective value in the latter case. As per the present formulation of the problem, given by Eq. (15), in fact the solution of Deb and Goyal [10] becomes infeasible by violating constraint $g_4(\boldsymbol{x})$ (constraints values as per the formulation of Eq. (15) are given in parentheses). However, the variation could not be analyzed as the complete formulation of the problem is not provided by Deb and Goyal [10]. It is to be mentioned that, in the 30 runs of the proposed PSO, the solutions are found varying in the values of l only, which vary in the range of [1.782102,1.784597] in. with the corresponding variation of the objective values in the range of [1.955301,1.955645]. On the other hand, the solution of the PSO of Kennedy and Eberthart [16] is quite different, giving the worst objective value of 2.025363.

Table 9
Optimal solutions for the welding beam design problem.

Variables/ functions	Deb and Goyal [10]	PSO of Kennedy and Eberthart [16]	Proposed PSO
x_1	1	1	1
x_2	0	0	0
h (in.)	0.1875	0.125000	0.187500
t (in.)	8.2500	8.312500	8.250000
b (in.)	0.2500	0.250000	0.250000
l (in.)	1.6849	4.115994	1.782103
$-g_1(\boldsymbol{x})$	380.1660 (380.165289)	823.901860	380.165289
$-g_2(\boldsymbol{x})$	402.0473 (402.047213)	435.707814	402.047213
$-g_3(\boldsymbol{x})$	0.2346 (0.234362)	0.234712	0.234362
$-g_4(\boldsymbol{x})$	0.1621 (−394.196461)	363.700864	0.004258
$f(\boldsymbol{x})$	1.9422 (infeasible)	2.025363	1.955301

In the case of computational cost, the numbers of function evaluation are found varying in the range of [19584,127848] over the 30 runs of the proposed PSO, with the mean, median and standard deviation of 69956.8, 64004 and 175.9, respectively. The number of function evaluations required by the PSO of Kennedy and Eberthart [16] in obtaining its best solution was 189800, which is also quite higher than those required by the proposed PSO.

6. Conclusions

The success of particle swarm optimization (PSO) to different real-valued problems has encouraged to develop some integer-coded versions of PSO for dealing with integer and discrete variables of a problem. However, most of such versions actually convert a real-valued solution of a real-coded PSO into an integer-valued solution by applying some random schemes. Only a limited number of works are found in which some problem-specific versions of integer-coded PSO are proposed. A novel version of PSO of general form is proposed here, which can work directly with real, integer, and discrete variables of a problem without any conversion. In order to improve the performance of the proposed PSO, a local search scheme is also applied to the global best particle of the swarm. The effectiveness of the proposed PSO is demonstrated through its successful application to some mechanical design problems of different nature. Apart from comparing the numerical solutions of the proposed PSO with those obtained by other methods, these are also compared with those obtained by using the binary-coded PSO of Kennedy and Eberthart [16]. It is observed that the performance of the PSO of Kennedy and Eberthart [16] falls with increasing complexity of the problems, while the proposed PSO is found unaffected by their complexities. Further research may be carried out to exploit the potentialities of the proposed PSO by applying it to other design problems. Moreover, attempts may also be made for further improvement on its performance.

Acknowledgments

The work is carried out under the first author's post-doctoral fellowship grant (SFRH/BPD/34482/2006) offered by the *Fundaç ão para a Ciência e a Tecnologia* (FCT), *Ministério da Ciência, Tecnologia e Ensino Superior*, Portugal.

References

[1] D. Anghinolfi, M. Paolucci, A new discrete particle swarm optimization approach for the single-machine total weighted tardiness scheduling problem with sequence dependent setup times, European Journal of Operational Research 193 (2009) 73–85.

[2] J.S. Arora, Introduction to Optimum Design, Elsevier, San Diego, CA, 2004.
[3] J.L. Chen, Y.C. Tsao, Optimal design of machine elements using genetic algorithms, Journal of Chinese Society of Mechanical Engineers 14 (2) (1993) 193–199.
[4] C.A.C. Coello, Constraint-handling using an evolutionary multiobjective optimization technique, Civil Engineering and Environmental Systems 17 (2000) 319–346.
[5] D. Datta, C.M. Fonseca, K. Deb, A multi-objective evolutionary algorithm to exploit the similarities of resource allocation problems, Journal of Scheduling 11 (6) (2008) 405–419.
[6] K. Deb, Optimal design of a welded beam structure via genetic algorithms, AIAA Journal 29 (11) (1991) 2013–2015.
[7] K. Deb, GeneAS: a robust optimal design technique for mechanical component design, in: Evolutionary Algorithms in Engineering Applications, Springer, Berlin, 1997, pp. 497–514.
[8] K. Deb, An efficient constraint handling method for genetic algorithms, Computer Methods in Applied Mechanics and Engineering 186 (2000) 311–338.
[9] K. Deb, Multi-Objective Optimization using Evolutionary Algorithms, John Wiley & Sons, Ltd., Chichester, England, 2001.
[10] K. Deb, M. Goyal, Optimizing engineering designs using a combined genetic search, in: Sixth International Conference on Genetic Algorithms, Morgan Kauffman Publishers, 1995, pp. 521–528.
[11] K. Deb, A. Srinivasan, Innovization: innovating design principles through optimization, in: Eighth annual conference on Genetic and Evolutionary Computation (GECCO'06), 2006, pp. 1629–1636.
[12] C. Guo, J. Hu, B. Ye, Y. Cao, Swarm intelligence for mixed-variable design optimization, Journal of Zhejiang University Science 5 (7) (2004) 851–860.
[13] B.K. Kannan, S.N. Kramer, An augmented Lagrange multiplier based method for mixed integer discrete continuous optimization and its application to mechanical design, ASME Transactions on Mechanical Design 116 (1995) 405–411.
[14] A.H. Kashan, B. Karimi, A discrete particle swarm optimization algorithm for scheduling parallel machines, Computers and Industrial Engineering 56 (2009) 216–223.
[15] J. Kennedy, R. Eberthart, Particle swarm optimization, in: IEEE International Conference on Neural Networks, Vol. 4., Perth, Australia, 1995, pp. 1942–1948.
[16] J. Kennedy, R. Eberthart, A discrete binary version of the particle swarm optimizer, IEEE Conference on Computational Cybernetics and Simulation 5 (1997) 4104–4108.
[17] J. Lampinen, I. Zelinka, Mixed-integer-discrete-continuous optimization by differential evolution. Part 2. A practical example, in: P. Ošmera (Ed.), Fifth International Mendel Conference on Soft Computing (MENDEL'99), Brno, Czech Republic, 1999, pp. 77–81.
[18] E.C. Laskari, K.E. Parsopoulos, M.N. Vrahatis, Particle swarm optimization for integer programming, in: IEEE Congress on Evolutionary Computation (CEC'02), Honolulu, HI, 2002, pp. 1582–1587.
[19] C.-J. Liao, C.-T. Tseng, P. Luarn, A discrete version of particle swarm optimization for flowshop scheduling problems, Computers and Operations Research 34 (2007) 3099–3111.
[20] S.S. Lin, C. Zhang, H.P. Wang, On mixed-discrete nonlinear optimization problems: a comparative study, Engineering Optimization 23 (1995) 287–300.
[21] D.S. Liu, K.C. Tan, S.Y. Huang, C.K. Goh, W.K. Ho, On solving multiobjective bin packing problems using evolutionary particle swarm optimization, European Journal of Operational Research 190 (2008) 357–382.
[22] H.T. Loh, P.Y. Papalambros, A sequential linearization approach for solving mixed-discrete nonlinear design optimization problems, ASME Journal of Mechanical Design 113 (1991) 325–334.
[23] K.M. Ragsdell, D.T. Phillips, Optimal design of a class of welded structures using geometric programming, ASME Journal of Engineering for Industries 98 (3) (1976) 1021–1025.
[24] E. Sandgren, Nonlinear integer and discrete programming in mechanical design optimization, ASME Transactions on Mechanical Design 112 (2) (1990) 223–229.
[25] D.Y. Sha, C.-Y. Hsu, A hybrid particle swarm optimization for job shop scheduling problem, Computers and Industrial Engineering 51 (2006) 791–808.
[26] F.M. Tasgetiren, Y.C. Liang, M. Sevkli, G. Gencyilmaz, A particle swarm optimization algorithm for makespan and total flowtime minimization in the permutation flowshop sequencing problem, European Journal of Operational Research 177 (2007) 1930–1947.
[27] S.-J. Wu, P.-T. Chow, Genetic algorithms for nonlinear mixed discrete-integer optimization problems via meta-genetic parameter optimization, Engineering Optimization 24 (2) (1995) 137–159.
[28] T. Yokota, T. Taguchi, M. Gen, A solution method for optimal cost problem of welded beam by using genetic algorithms, Computers and Industrial Engineering 37 (1999) 379–382.
[29] C. Zhang, H.P. Wang, Mixed-discrete nonlinear optimization with simulated annealing, Engineering Optimization 21 (1993) 277–291.